

DFTL Experiment in FEMU

2026. 08.11.

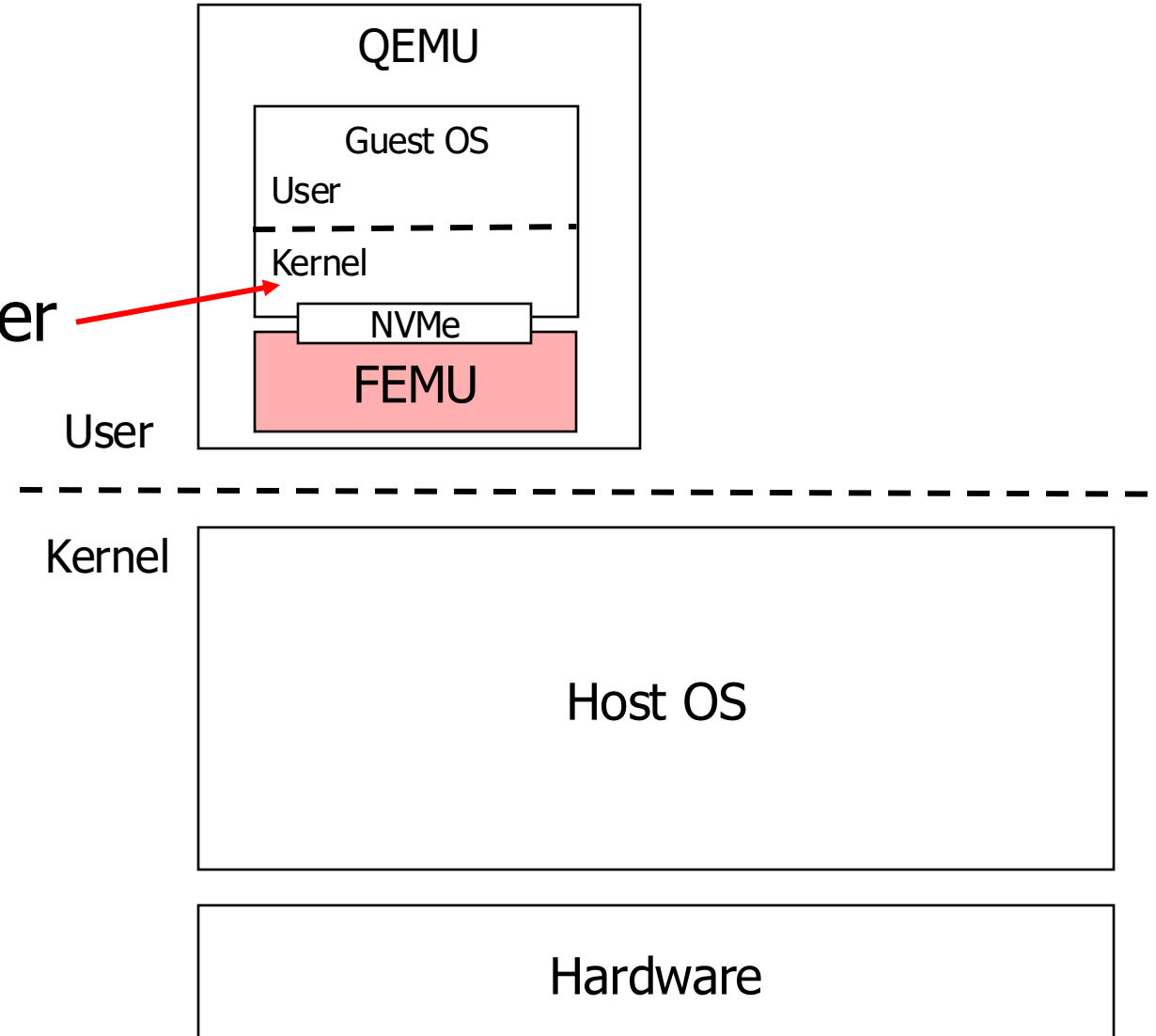
Presentation by Sung JinWook, Jung Huichan
sungjw0408@dankook.ac.kr, a011214@dankook.ac.kr

Contents

1. What is FEMU?
2. FTL Implementation in FEMU
3. Reproducing DFTL in FEMU

1. What is FEMU?

- Based on QEMU
- Interact with Virtual NVMe Driver
- More Cheap & Accurate
NVMe SSD Emulator



1.1. Why is FEMU Effective?

- Simulator
 - ex. DiskSim+SSD, FlashSim
 - Fast & Cheap
 - Trace-driven FTL Internal Simulation
→ FTL Research Only

1.1. Why is FEMU Effective?

■ Simulator

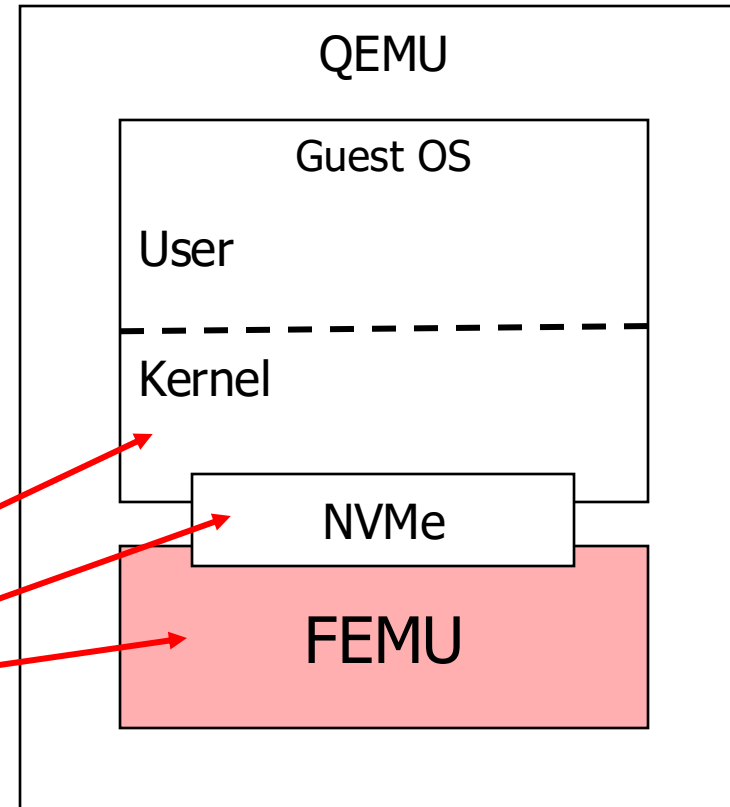
- ex. DiskSim+SSD, FlashSim
- Fast & Cheap
- **Trace-driven** FTL Internal Simulation
→ **FTL Research Only**

■ FEMU

- Full-stack Research

- ✓ **Kernel** Research
- ✓ **Interface** Research
- ✓ **FTL** Research

Ref: FAST '18 Presentation of "The CASE of FEMU: Cheap, Accurate, Scalable and Extensible Flash Emulator", Huaicheng Li, 2018

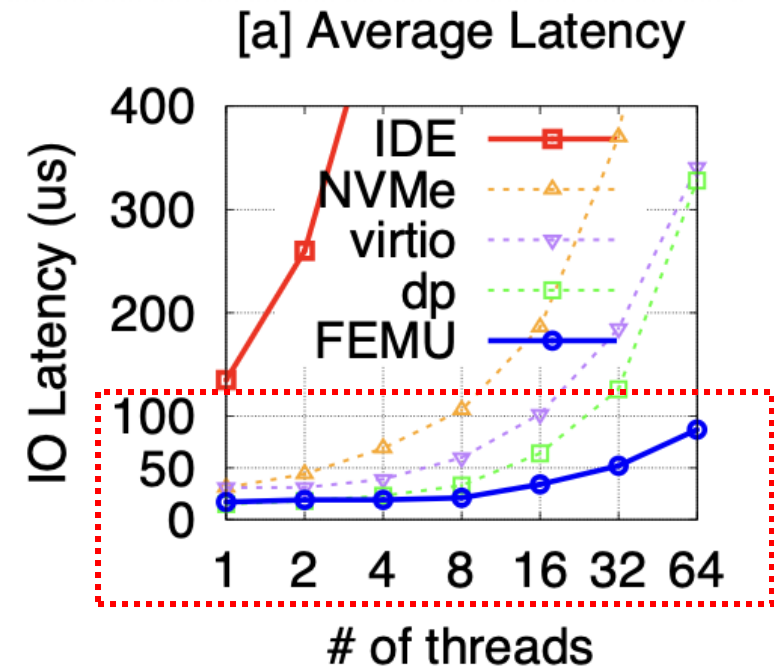


1.1. Why is FEMU Effective?

- Emulator
 - Virtual Block Device: Guest OS's **Virtual SSD**
 - ex. VSSIM, LightNVM's QEMU
 - Cheap & Full-stack Research
 - **Poor Latency & Accuracy**

1.1. Why is FEMU Effective?

- Emulator
 - Virtual Block Device: Guest OS's **Virtual SSD**
 - ex. VSSIM, LightNVM's QEMU
 - Cheap & Full-stack Research
 - **Poor Latency & Accuracy**
- FEMU
 - **Low Latency** in Simultaneous I/O Multi-threads
 - If Emulates $80\mu\text{s}$ Lat. SSD Using 32 Threads?
 - **(Baseline) $52\mu\text{s}$ + (Delay Injection) $28\mu\text{s}$**



(Each Thread:
4KB Random Read)

Ref: "The CASE of FEMU: Cheap,
Accurate, Scalable and Extensible
Flash Emulator", Huaicheng Li, 2018

1.1. Why is FEMU Effective?

- Hardware Platform
 - Install Firmware on **Physical SSD Board**
 - ex. OpenSSD, OpenChannelSSD
 - **Accurate** & Full-stack Research
 - Expensive & Wear-out Problem



Ref: OpenSSD, Daisy, <https://www.crz-tech.com/crz/os/daisy/>

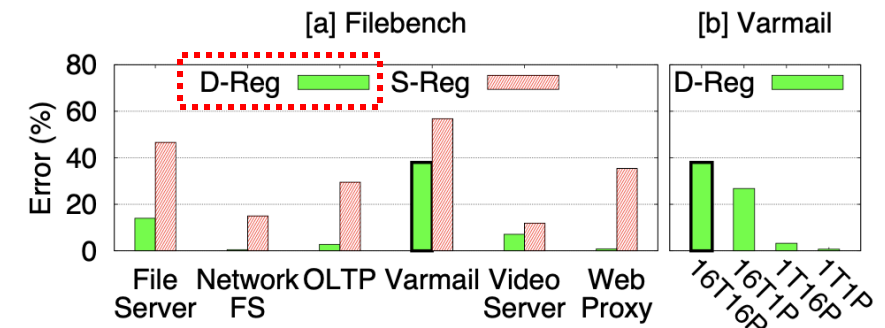
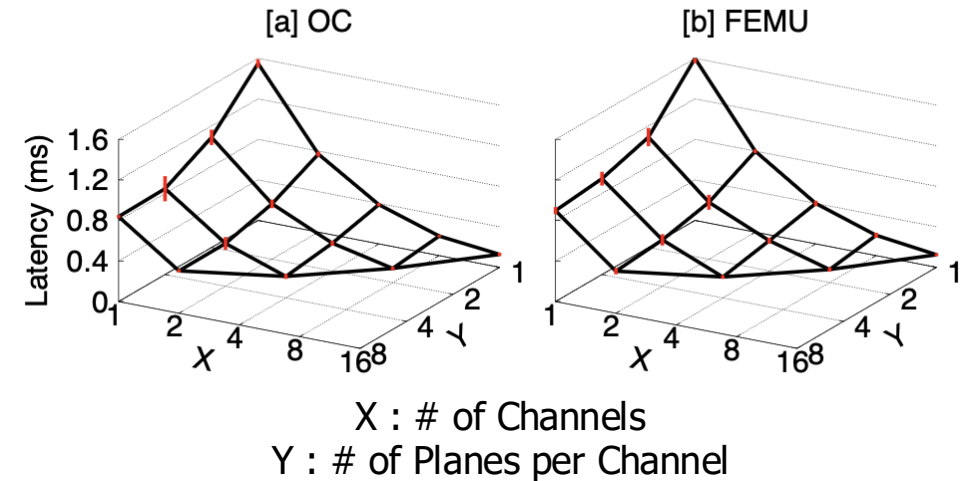
1.1. Why is FEMU Effective?

■ Hardware Platform

- Install Firmware on **Physical SSD Board**
- ex. OpenSSD, OpenChannelSSD
- **Accurate** & Full-stack Research
- Expensive & Wear-out Problem

■ FEMU

- **Similar Latency** with **OCSSD**
- **Lower Latency Error** than Legacy Model

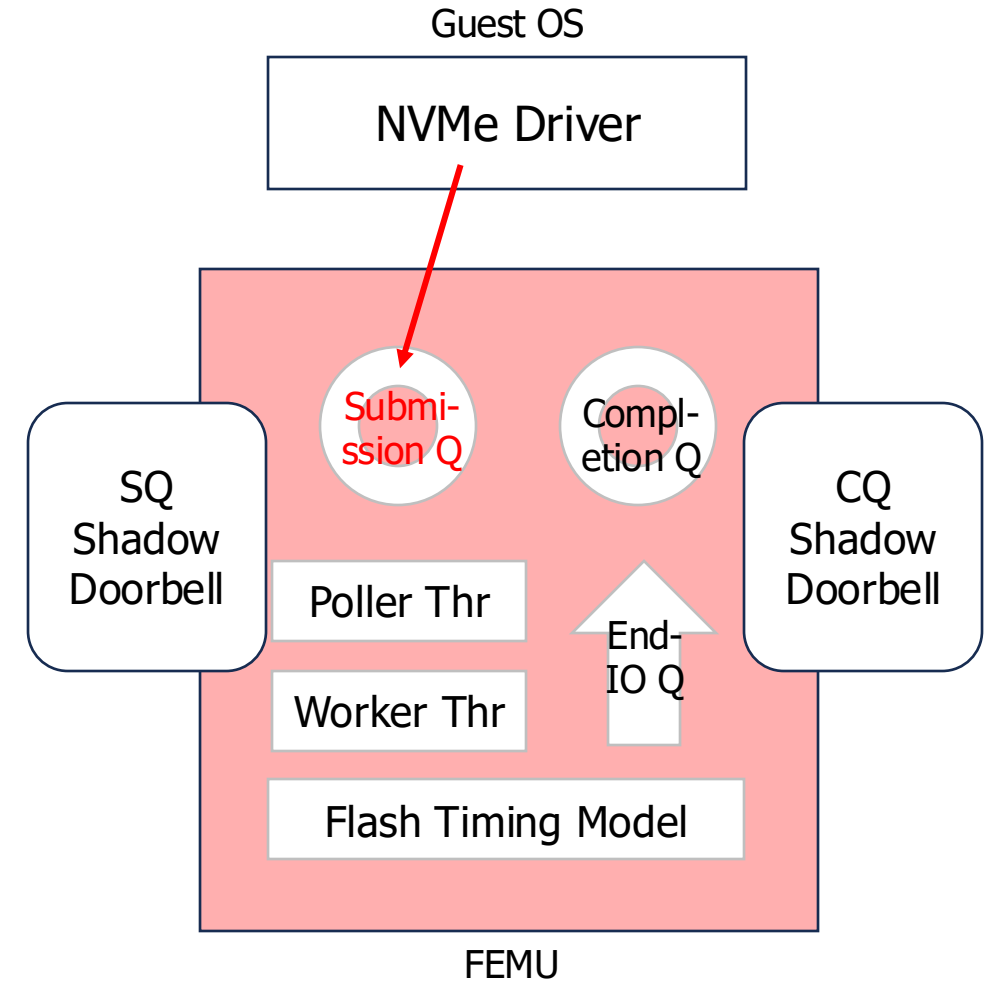


$$\text{Error}(\%) = |\text{Latency_femu} - \text{Latency_oc}| / \text{Latency_oc}$$

1.2. How does FEMU Operate?

1) NVMe Driver to FEMU

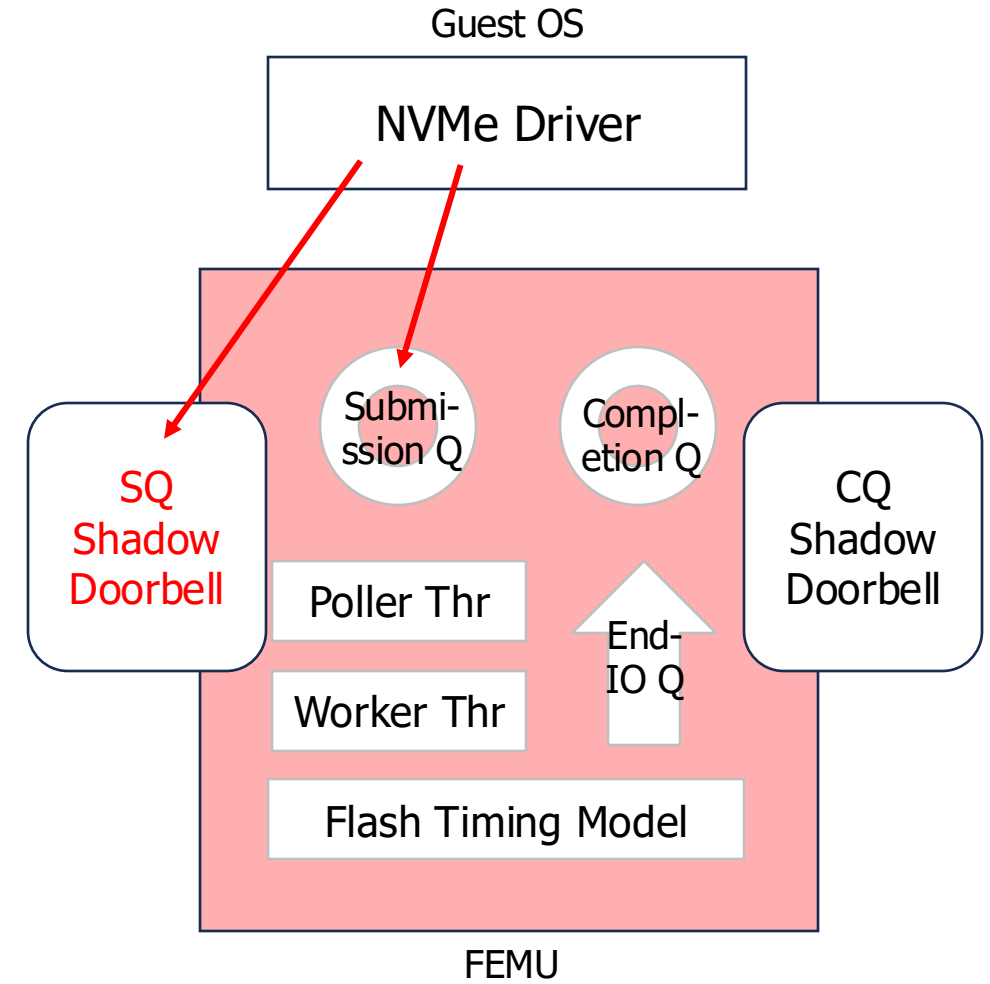
- Submission Queue
 - Stores I/O Commands



1.2. How does FEMU Operate?

1) NVMe Driver to FEMU

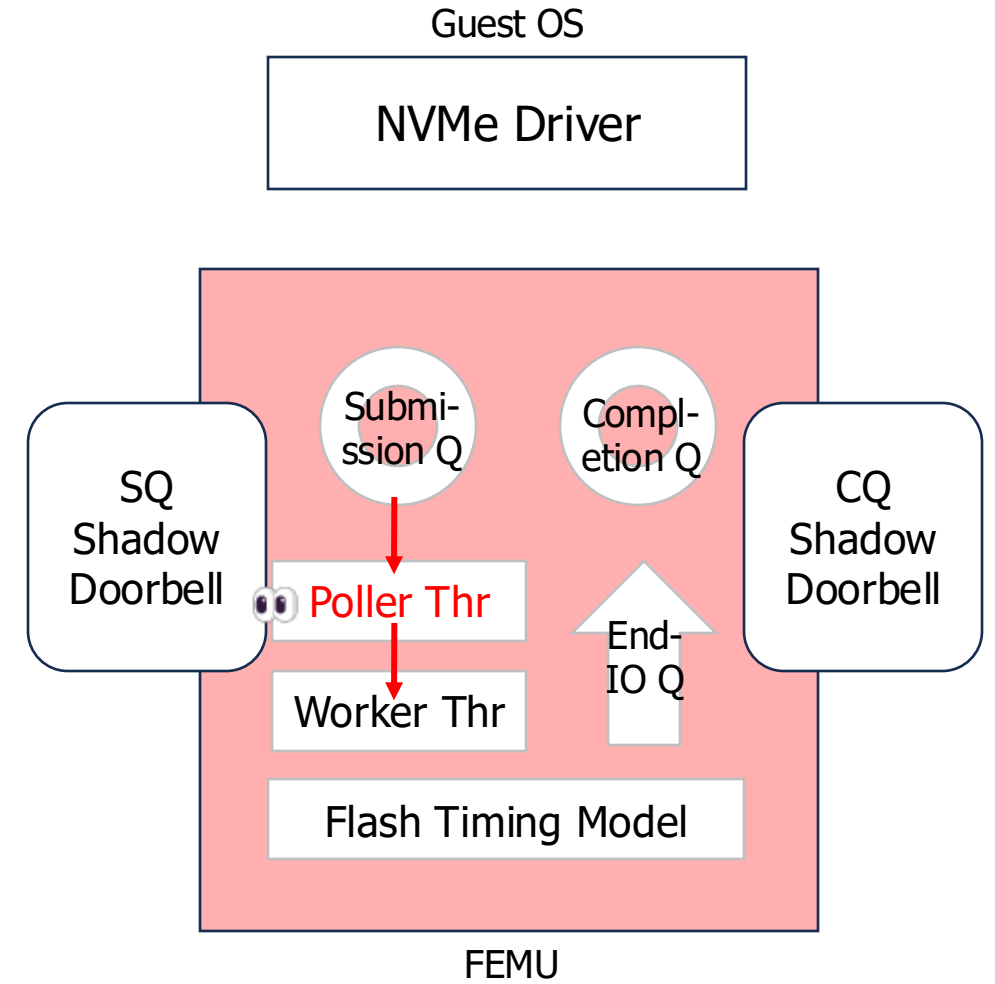
- Submission Queue
 - Stores I/O Commands
- SQ Shadow Doorbell
 - **Memory** for SQ
 - **Shared with Guest OS & QEMU**



1.2. How does FEMU Operate?

2) Threads in FEMU Work

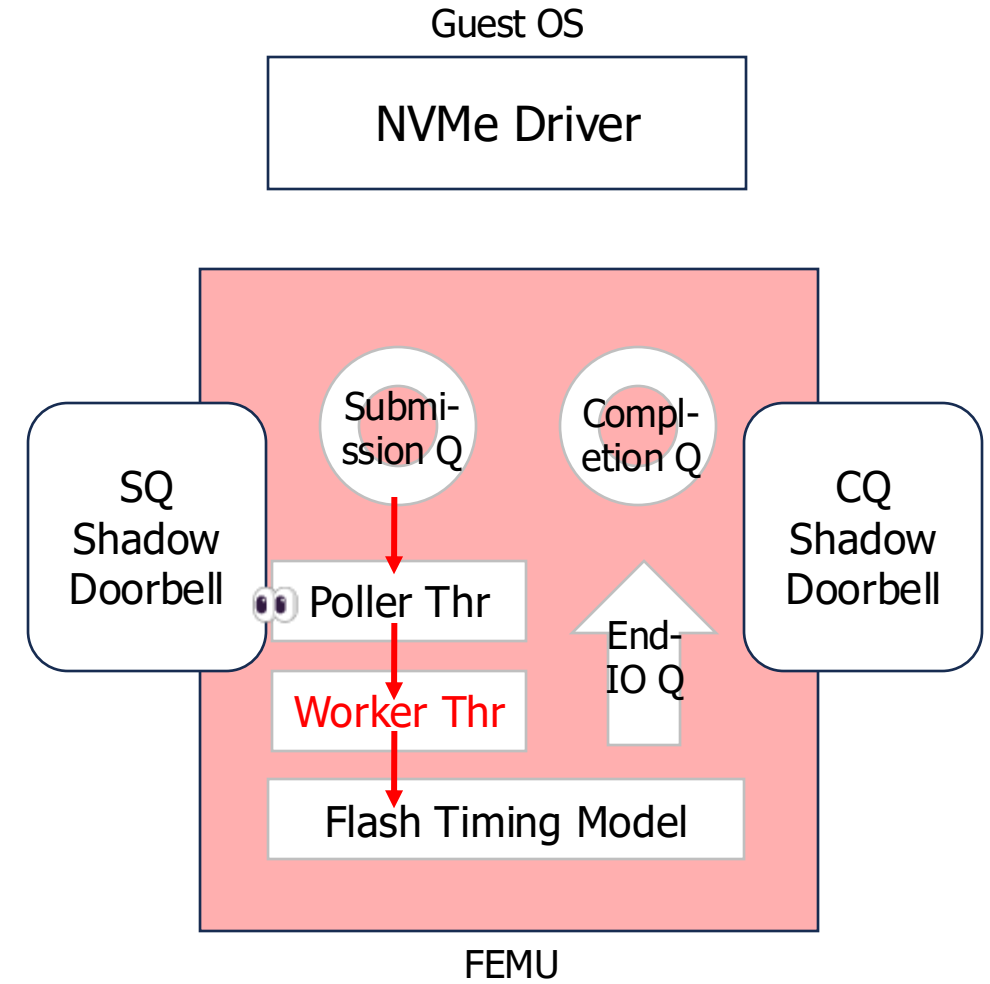
- Poller Thread
 - Detects & **Brings I/O Commands**
 - Delivers Commands to [FTL Worker Thread]



1.2. How does FEMU Operate?

2) Threads in FEMU Work

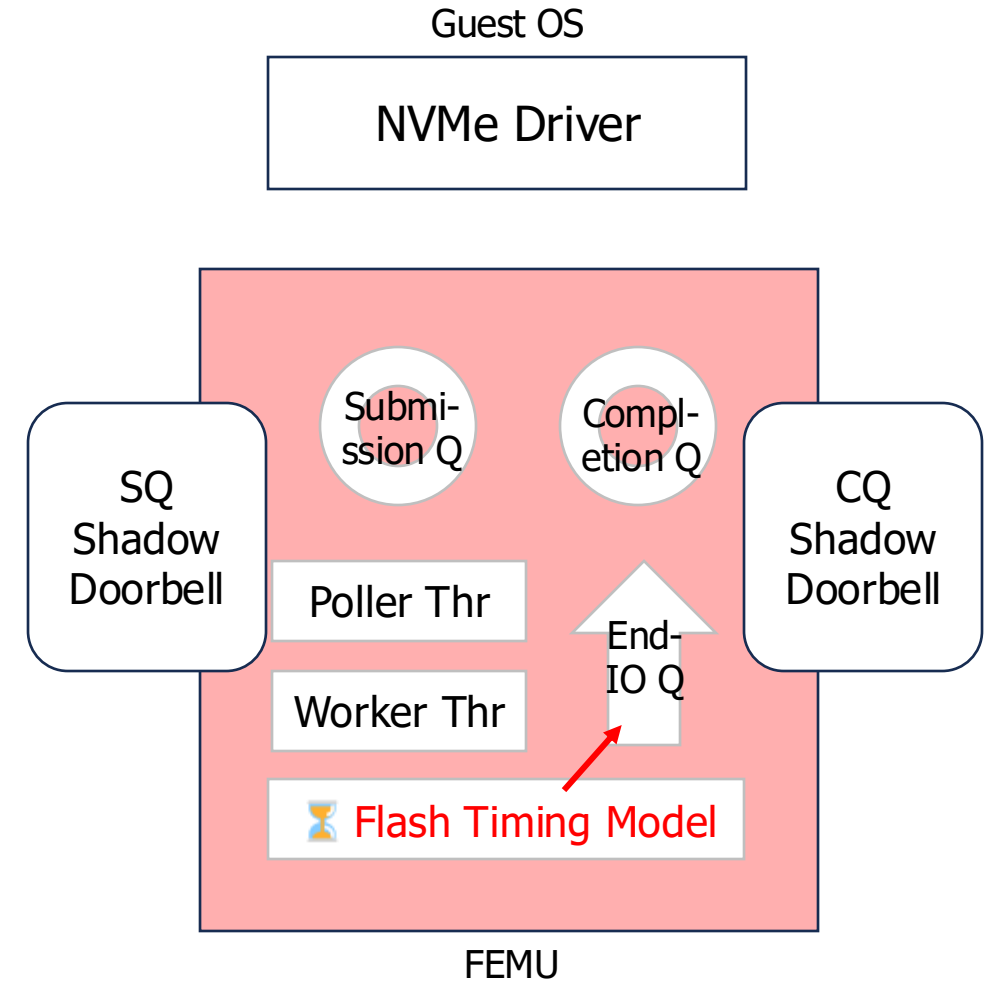
- Poller Thread
 - Detects & **Brings I/O Commands**
 - Delivers Commands to [FTL Worker Thread]
- FTL Worker Thread
 - Executes **Address Translation & GC**
 - Delivers Results to [Flash Timing Model]



1.2. How does FEMU Operate?

3) Calculates Latency

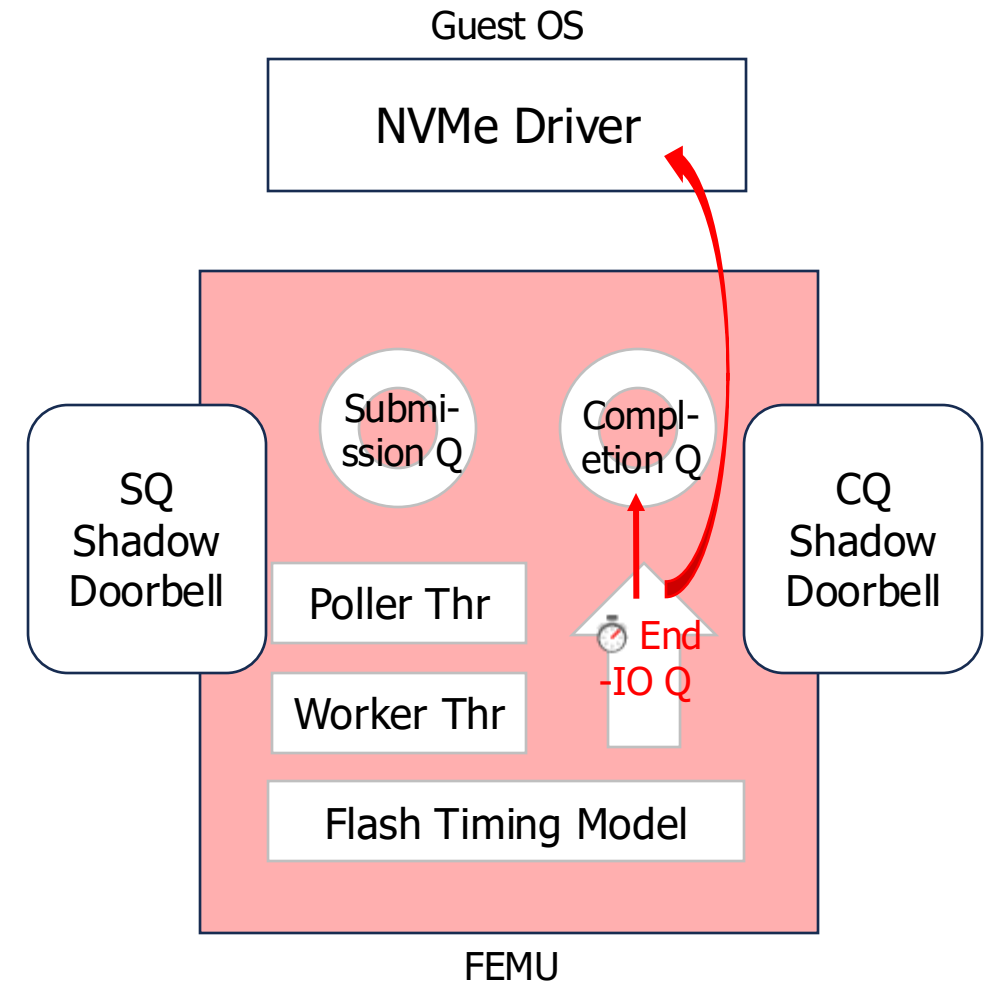
- Flash Timing Model
 - Calculates **Read/Write/Erase Latency** per Unit
 - Reflects **Hierarchy of NAND Flash**
(channel > chip > die > plane > block > page)
 - Puts Results to [End-IO Queue]



1.2. How does FEMU Operate?

4) FEMU to NVMe Driver

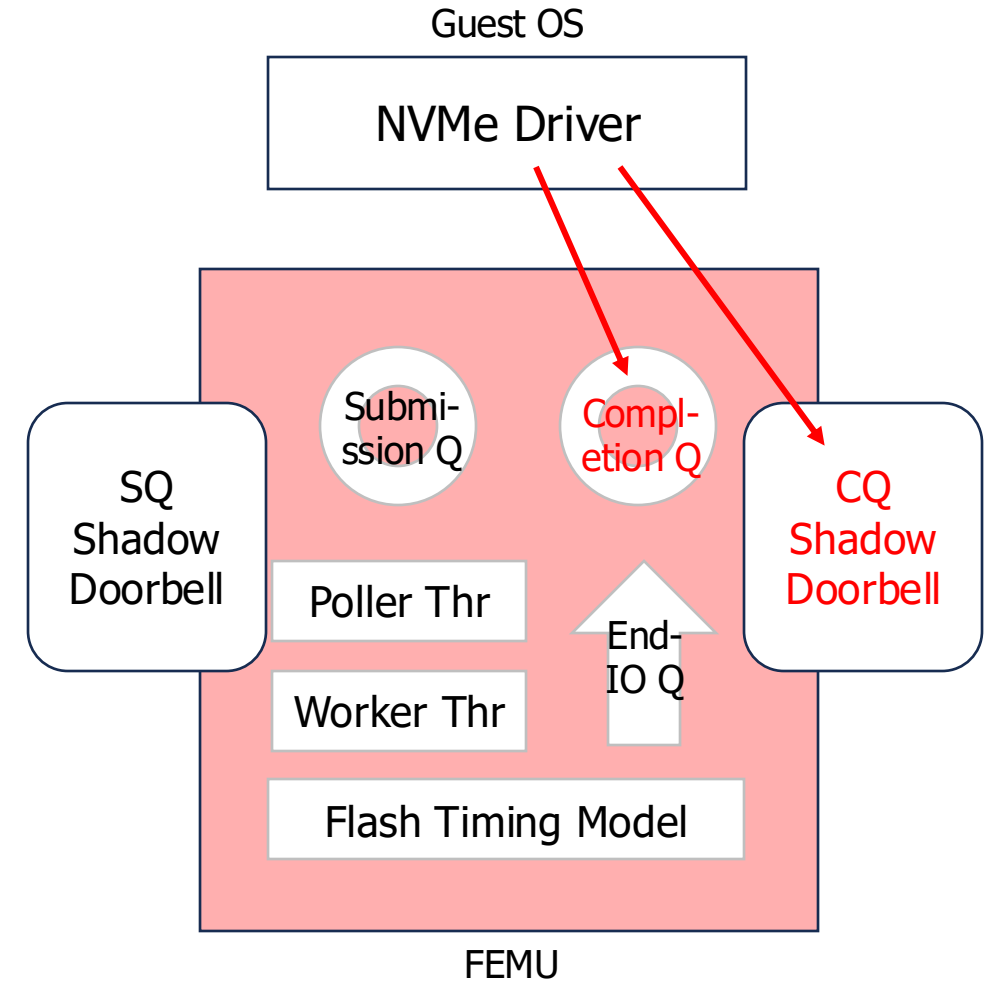
- End-IO Queue/Thread
 - **Waits Until Target Time**(ex. $80\mu s$) Comes by Delay Injection(ex. $28\mu s$)
 - Updates [Completion Queue]
 - Delivers Interrupt to NVMe Driver
- Completion Queue



1.2. How does FEMU Operate?

5) Confirmation of NVMe Driver

- Completion Queue
 - Cleared by NVMe Driver
- CQ Shadow Doorbell
 - **Memory** for CQ
 - **Shared with Guest OS & QEMU**



2. FTL Implementation in FEMU

- Multiple Models of FEMU
 - Blackbox SSD Mode
 - OS Doesn't Know Physical Address & FTL Inner Logic
 - OS Just Knows Standard Interface (ex. Read, Write, Trim)
 - Like Commercialized SSD
 - Whitebox SSD Mode
 - OS Knows & Cares PA & FTL
 - Like OCSSD

2. FTL Implementation in FEMU

FEMU / hw / femu / bbssd /

huaicheng femu/bbssd: harden the dftl mapping cc

Name

..

bb.c

ftl-cache.c

ftl-datapath.c

ftl-exp.c

ftl-fdp.c

ftl-geom.c

ftl-internal.h

ftl-line-gc.c

ftl-map-cmt.c

ftl-map.c

ftl-media.c

ftl.c

ftl.h

1) FTL Initialization

- `ftl-geom.c`, `ftl.c`, `bb.c`

2) Address Mapping Management

- `ftl-internal.h`, `ftl-map.c`

3) Data Allocation & I/O Processing

- `ftl-datapath.c`, `ftl-media.c`

4) GC & Wear-leveling

- `ftl-line-gc.c`

2.1. FTL Initialization

- ftl-geom.c
 - Configures Parameter of SSD (*ssdparams*)
 - Size of Sector/Page/...
 - GC Threshold

```
ftl-geom.c 3 × ftl.c 2 bb.c 2
ftl-geom.c > ssd_init_params(ssdparams *, FemuCtrl *)

21 void ssd_init_params(struct ssdparams *spp, FemuCtrl *n)
22 {
23     spp->secsz = n->bb_params.secsz; // 512
24     spp->secs_per_pg = n->bb_params.secs_per_pg; // 8
25     spp->pgs_per_blk = n->bb_params.pgs_per_blk; //256
26     spp->blks_per_pl = n->bb_params.blks_per_pl; /* 256 16GB */
27     spp->pls_per_lun = n->bb_params.pls_per_lun; // 1
28     spp->luns_per_ch = n->bb_params.luns_per_ch; // 8
29     spp->nchs = n->bb_params.nchs; // 8
```

```
ftl-geom.c 3 × ftl.c 2 bb.c 2
ftl-geom.c > ssd_init_params(ssdparams *, FemuCtrl *)

21 void ssd_init_params(struct ssdparams *spp, FemuCtrl *n)
63     spp->gc_thres_pcent = n->bb_params.gc_thres_pcent/100.0;
64     spp->gc_thres_lines = (int)((1 - spp->gc_thres_pcent) * spp->tt_lines);
65     spp->gc_thres_pcent_high = n->bb_params.gc_thres_pcent_high/100.0;
66     spp->gc_thres_lines_high = (int)((1 - spp->gc_thres_pcent_high) * spp->tt_lines);
67     spp->enable_gc_delay = true;
```

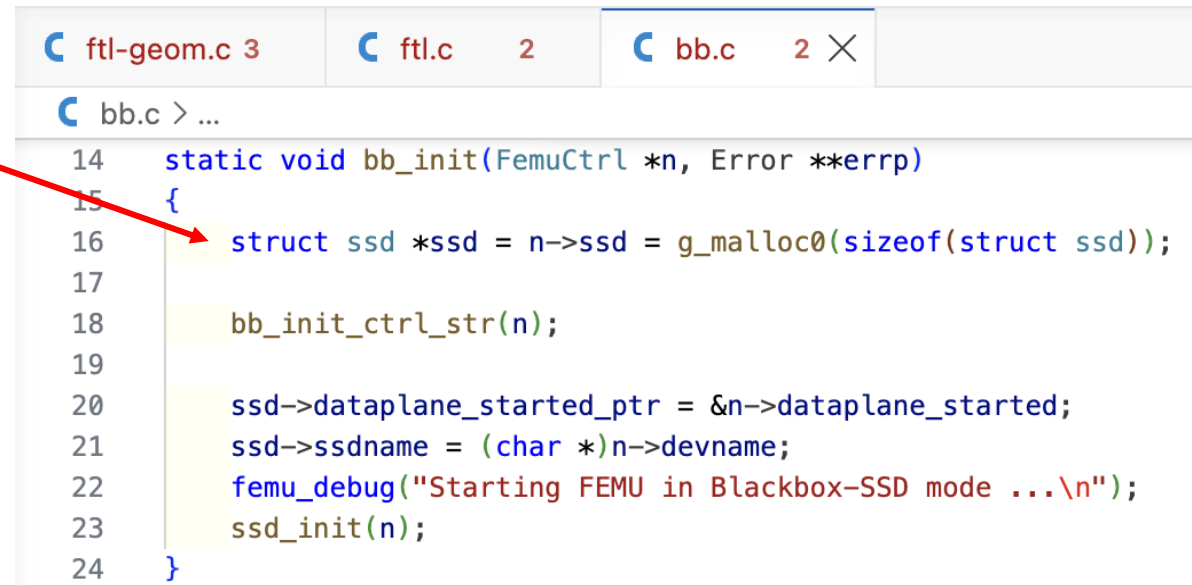
2.1. FTL Initialization

- `ftl.c`
 - Manages Initialization of FTL (`ssd_init()`)
 - Mapping Table (`maptbl`, `rmap`)
 - Block (Lines)

```
ftl-geom.c 3  ftl.c 2 X  bb.c 2
ftl.c > ssd_init(FemuCtrl *)
135 void ssd_init(FemuCtrl *n)
136 {
137     struct ssd *ssd = n->ssd;
138     struct ssdparams *spp = &ssd->spp;
139
140     ftl_assert(ssd);
141     ssd->n = n;
142
143     /* read the data-remanence experiment env vars once (debug only, off by def
144     exp_load_cfg();
145
146     ssd_init_params(spp, n);
147
148     /* initialize ssd internal layout architecture */
149     ssd->ch = g_malloc0(sizeof(struct ssd_channel) * spp->nchs);
150     for (int i = 0; i < spp->nchs; i++) {
151         ssd_init_ch(&ssd->ch[i], spp);
152     }
153
154     /* configure the NAND media-layer timing (reads spp, points at ssd->ch) */
155     bb_nand_media_init(ssd);
156
157     /* initialize maptbl */
158     ssd_init_maptbl(ssd);
159
160     /* initialize rmap */
161     ssd_init_rmap(ssd);
162
163     /* initialize all the lines */
164     ssd_init_lines(ssd);
```

2.1. FTL Initialization

- `bb.c`
 - Manages Initialization of BBSSD Controller (`bb_init()`)



```
ftl-geom.c 3  ftl.c 2  bb.c 2 X
bb.c > ...
14 static void bb_init(FemuCtrl *n, Error **errp)
15 {
16     struct ssd *ssd = n->ssd = g_malloc0(sizeof(struct ssd));
17
18     bb_init_ctrl_str(n);
19
20     ssd->dataplane_started_ptr = &n->dataplane_started;
21     ssd->ssdname = (char *)n->devname;
22     femu_debug("Starting FEMU in Blackbox-SSD mode ...\\n");
23     ssd_init(n);
24 }
```

2.2. Address Mapping Management

- ftl-internal.h ★
 - *maptbl[]*: MT that LPN $\rightarrow$ PPA
 - *rmap[]*: MT that PPA $\rightarrow$ LPN
 - *get_maptbl_ent(LPN)*
 - Returns PPA
 - *set_maptbl_ent(LPN)*
 - Updates Mapping Table

```
h ftl-internal.h 1 X C ftl-map.c
h ftl-internal.h > valid_ppa(ssd *, ppa *)
1  #ifndef __FEMU_BBSSD_FTL_INTERNAL_H    #include 오류가 검색되었습니다. includePath를 업데이트
74  /* mapping accessors (maptbl / rmap) -- hot-path inlines */
75  static inline struct ppa get_maptbl_ent(struct ssd *ssd, uint64_t lpn)
76  {
77      return ssd->maptbl[lpn];
78  }
79
80  static inline void set_maptbl_ent(struct ssd *ssd, uint64_t lpn, struct ppa *ppa)
81  {
82      ftl_assert(lpn < ssd->sp.tt_pgs);
83      ssd->maptbl[lpn] = *ppa;
84  }
```

2.2. Address Mapping Management

- ftl-internal.h ★
 - *valid_ppa(PPA)*
 - Calculates PPA is Right
 - Executes **Right After** *get_maptbl_ent(LPN)*
: ftl-datapath.c

```
h ftl-internal.h 1 X ftl-map.c
h ftl-internal.h > valid_ppa(ssd *, ppa *)
1  #ifndef __FEMU_BBSSD_FTL_INTERNAL_H    #include 오류가 검색되었습니다. includePath를 업데이트
12 static inline bool valid_ppa(struct ssd *ssd, struct ppa *ppa)
13 {
14     struct ssdparams *spp = &ssd->sp;
15     int ch = ppa->g.ch;
16     int lun = ppa->g.lun;
17     int pl = ppa->g.pl;
18     int blk = ppa->g.blk;
19     int pg = ppa->g.pg;
20     int sec = ppa->g.sec;
21
22     if (ch >= 0 && ch < spp->nchs && lun >= 0 && lun < spp->luns_per_ch && pl >=
23         0 && pl < spp->pls_per_lun && blk >= 0 && blk < spp->blks_per_pl && pg
24         >= 0 && pg < spp->pgs_per_blk && sec >= 0 && sec < spp->secs_per_pg)
25         return true;
26
27     return false;
28 }
```

2.2. Address Mapping Management

- `ftl-internal.h` ★
 - *`maptbl[]`*: MT that LPN $\rightarrow$ PPA
 - *`rmap[]`*: MT that PPA $\rightarrow$ LPN
 - *`get_rmap_ent(PPA)`*
 - Returns LPN
 - *`set_rmap_ent(LPN, PPA)`*
 - Updates Mapping Table
 - *`ppa2pgidx(PPA)`*
 - Calculates PPA to Page Index

```
h ftl-internal.h 1 X ftl-map.c
h ftl-internal.h > valid_ppa(ssd *, ppa *)
1  #ifndef __FEMU_BBSSD_FTL_INTERNAL_H    #include 오류가 검색되었습니다. includePath를 업데이
86  static inline uint64_t ppa2pgidx(struct ssd *ssd, struct ppa *ppa)
87  {
88      struct ssdparams *spp = &ssd->sp;
89      uint64_t pgidx;
90
91      pgidx = ppa->g.ch * spp->pgs_per_ch + \
92             ppa->g.lun * spp->pgs_per_lun + \
93             ppa->g.pl * spp->pgs_per_pl + \
94             ppa->g.blk * spp->pgs_per_blk + \
95             ppa->g.pg;
96
97      ftl_assert(pgidx < spp->tt_pgs);
98
99      return pgidx;
100 }
101
102 static inline uint64_t get_rmap_ent(struct ssd *ssd, struct ppa *ppa)
103 {
104     uint64_t pgidx = ppa2pgidx(ssd, ppa);
105
106     return ssd->rmap[pgidx];
107 }
108
109 /* set rmap[page_no(ppa)] -> lpn */
110 static inline void set_rmap_ent(struct ssd *ssd, uint64_t lpn, struct ppa *ppa)
111 {
112     uint64_t pgidx = ppa2pgidx(ssd, ppa);
113
114     ssd->rmap[pgidx] = lpn;
115 }
```

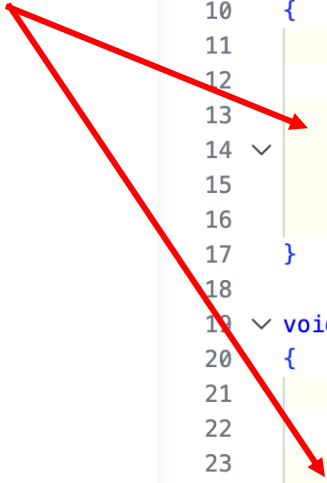

2.2. Address Mapping Management

- `ftl-internal.h` ★
 - *`maptbl[]`*: MT that LPN $\rightarrow$ PPA
 - *`rmap[]`*: MT that PPA $\rightarrow$ LPN
 - *`get_rmap_ent(PPA)`*
 - Returns LPN
 - *`set_rmap_ent(LPN, PPA)`*
 - Updates Mapping Table
 - *`ppa2pgidx(PPA)`*
 - Calculates PPA to Page Index

```
h ftl-internal.h 1 X C ftl-map.c
h ftl-internal.h > valid_ppa(ssd *, ppa *)
1  #ifndef __FEMU_BBSSD_FTL_INTERNAL_H    #include 오류가 검색되었습니다. includePath를 업데이트
86  static inline uint64_t ppa2pgidx(struct ssd *ssd, struct ppa *ppa)
87  {
88      struct ssdparams *spp = &ssd->sp;
89      uint64_t pgidx;
90
91      pgidx = ppa->g.ch * spp->pgs_per_ch + \
92             ppa->g.lun * spp->pgs_per_lun + \
93             ppa->g.pl * spp->pgs_per_pl + \
94             ppa->g.blk * spp->pgs_per_blk + \
95             ppa->g.pg;
96
97      ftl_assert(pgidx < spp->tt_pgs);
98
99      return pgidx;
100 }
101
102 static inline uint64_t get_rmap_ent(struct ssd *ssd, struct ppa *ppa)
103 {
104     uint64_t pgidx = ppa2pgidx(ssd, ppa);
105
106     return ssd->rmap[pgidx];
107 }
108
109 /* set rmap[page_no(ppa)] -> lpn */
110 static inline void set_rmap_ent(struct ssd *ssd, uint64_t lpn, struct ppa *ppa)
111 {
112     uint64_t pgidx = ppa2pgidx(ssd, ppa);
113
114     ssd->rmap[pgidx] = lpn;
115 }
```

2.2. Address Mapping Management

- ftl-map.c
 - Allocates Memory & Initializes of Mapping Table

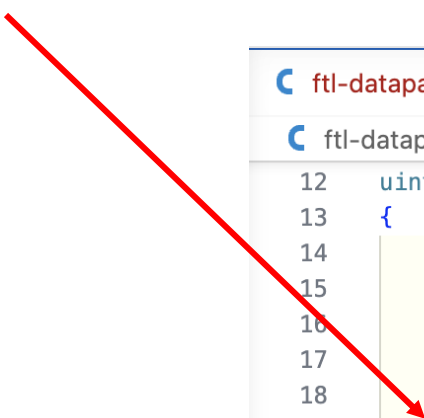


```
h ftl-internal.h 1  C ftl-map.c 3 X
C ftl-map.c > ...

9  void ssd_init_maptbl(struct ssd *ssd)
10 {
11     struct ssdparams *spp = &ssd->sp;
12
13     ssd->maptbl = g_malloc0(sizeof(struct ppa) * spp->tt_pgs);
14     for (int i = 0; i < spp->tt_pgs; i++) {
15         ssd->maptbl[i].ppa = UNMAPPED_PPA;
16     }
17 }
18
19 void ssd_init_rmap(struct ssd *ssd)
20 {
21     struct ssdparams *spp = &ssd->sp;
22
23     ssd->rmap = g_malloc0(sizeof(uint64_t) * spp->tt_pgs);
24     for (int i = 0; i < spp->tt_pgs; i++) {
25         ssd->rmap[i] = INVALID_LPN;
26     }
27 }
28
```

2.3. Data Allocation & I/O Processing

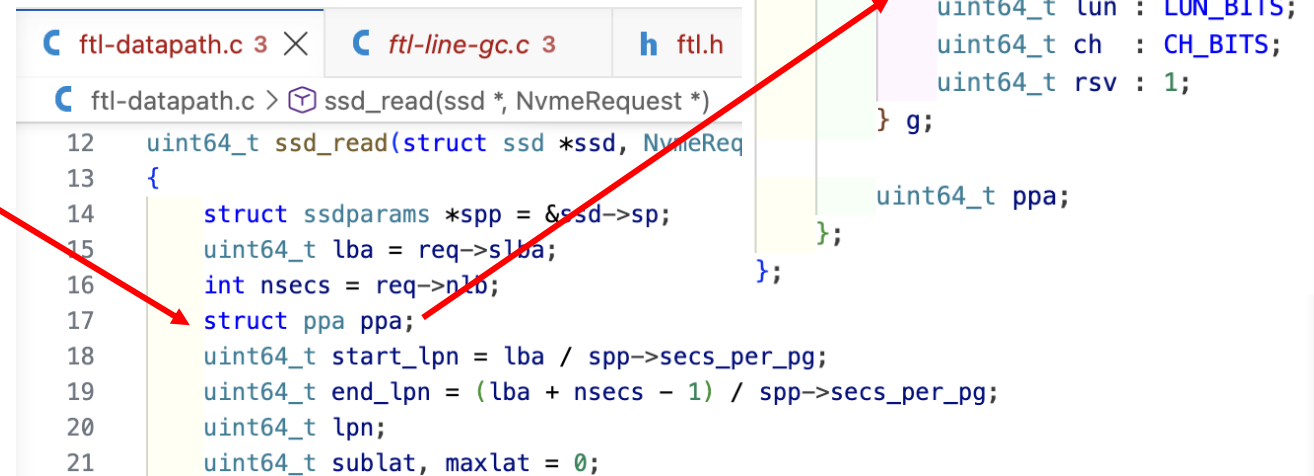
- ftl-datapath.c ★
 - *ssd_read()*
 - Defines *start_lpn*, *end_lpn* by *spp->secs_per_pg*



```
ftl-datapath.c 3 X  ftl-line-gc.c 3  h ftl.h 2  7
ftl-datapath.c > ssd_read(ssd *, NvmeRequest *)
12  uint64_t ssd_read(struct ssd *ssd, NvmeRequest *req)
13  {
14      struct ssdparams *spp = &ssd->sp;
15      uint64_t lba = req->slba;
16      int nsecs = req->nlb;
17      struct ppa ppa;
18      uint64_t start_lpn = lba / spp->secs_per_pg;
19      uint64_t end_lpn = (lba + nsecs - 1) / spp->secs_per_pg;
20      uint64_t lpn;
21      uint64_t sublat, maxlat = 0;
```

2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★
 - *ssd_read()*
 - Defines *start_lpn*, *end_lpn* by *spp->secs_per_pg*
 - Brings Struct of PPA
(Channel > Logic Unit Number
> Plane > Sector > Page > Block)



```
ftl-datapath.c 3 X  ftl-line-gc.c 3  h ftl.h

ftl-datapath.c > ssd_read(ssd *, NvmeRequest *)
12  uint64_t ssd_read(struct ssd *ssd, NvmeReq
13  {
14      struct ssdparams *spp = &ssd->sp;
15      uint64_t lba = req->slba;
16      int nsecs = req->nlb;
17      struct ppa ppa;
18      uint64_t start_lpn = lba / spp->secs_per_pg;
19      uint64_t end_lpn = (lba + nsecs - 1) / spp->secs_per_pg;
20      uint64_t lpn;
21      uint64_t sublat, maxlat = 0;

ftl.h
/* describe a physical page addr */
struct ppa {
    union {
        struct {
            uint64_t blk : BLK_BITS;
            uint64_t pg  : PG_BITS;
            uint64_t sec : SEC_BITS;
            uint64_t pl  : PL_BITS;
            uint64_t lun : LUN_BITS;
            uint64_t ch  : CH_BITS;
            uint64_t rsv : 1;
        } g;

        uint64_t ppa;
    };
};
```

2.3. Data Allocation & I/O Processing

■ ftl-datapath.c ★

■ *ssd_read()* - Each LPN:

1. Returns PPA
2. PPA Validation Check
3. Measures Latency & Calls Physical NAND Read

```
30      /* normal IO read path */
31      for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
32          /* dump the FEMU_DUMP_LPN page on every read (config read once at init) */
33          if (exp_dump_lpn_set && lpn == exp_dump_lpn)
34              femu_dbg_dump_lpn(ssd, lpn);
35
36          ppa = get_maptbl_ent(ssd, lpn);
37          if (!mapped_ppa(&ppa) || !valid_ppa(ssd, &ppa)) {
38              //printf("%s,lpn(%" PRIu64 ") not mapped to valid ppa\n", ssd->ssdname
39              //printf("Invalid ppa,ch:%d,lpn:%d,blk:%d,pl:%d,pg:%d,sec:%d\n",
40              //ppa.g.ch, ppa.g.lun, ppa.g.blk, ppa.g.pl, ppa.g.pg, ppa.g.sec);
41              continue;
42          }
43
44          struct nand_cmd srd;
45          srd.type = USER_IO;
46          srd.cmd = NAND_READ;
47          srd.stime = req->stime;
48          sublat = ssd_advance_status(ssd, &ppa, &srd);
49          maxlat = (sublat > maxlat) ? sublat : maxlat;
50      }
51
52      return maxlat;
53 }
```

2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★

- *ssd_write()*

- 0. If GC Needs

- Perform GC Before Write

```
ftl-datapath.c 3 ×  ftl-line-gc.c 3  h ftl.h 2
ftl-datapath.c > ssd_trim(ssd *, NvmeRequest *)
55  uint64_t ssd_write(struct ssd *ssd, NvmeRequest *req)
74      while (should_gc_high(ssd)) {
75          /* perform GC here until !should_gc(ssd) */
76          r = do_gc(ssd, true);
77          if (r == -1)
78              break;
79      }
```

2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★
 - *ssd_write()* - Each LPN:

1. Returns PPA
2. If PPA Already Exists
 - Marks PPA as Invalid
 - Updates RMT that PPA to INVALID_LPN

```
ftl-datapath.c 3 ×  ftl-line-gc.c 3  ftl.h 2
ftl-datapath.c > ssd_trim(ssd *, NvmeRequest *)
55  uint64_t ssd_write(struct ssd *ssd, NvmeRequest *req)
81      for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
82          ppa = get_maptbl_ent(ssd, lpn);
83          if (mapped_ppa(&ppa)) {
84              /* update old page information first */
85              if (exp_lpn_watched(lpn))
86                  EXP_LOG("[INVALIDATE:overwrite] lpn=%lu old " PPA_FMT "\n",
87                          lpn, PPA_ARG(&ppa));
88              mark_page_invalid(ssd, &ppa);
89              set_rmap_ent(ssd, INVALID_LPN, &ppa);
90          }
```

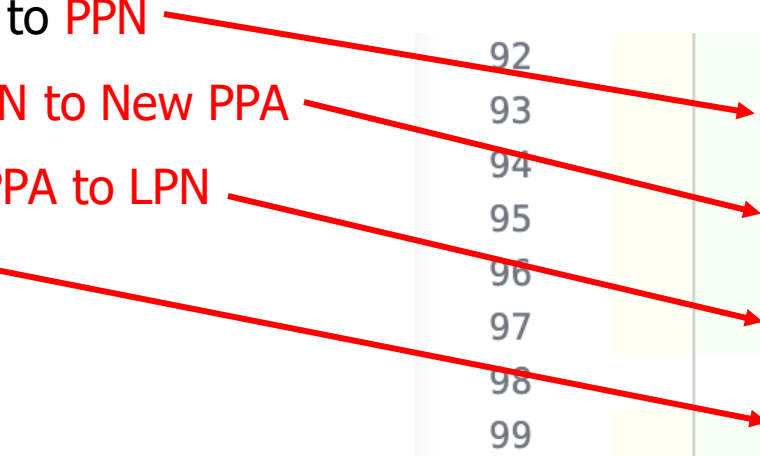
2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★

- *ssd_write()* - Each LPN:

3. Allocates New Page to PPA
4. Updates MT that LPN to New PPA
5. Updates RMT that PPA to LPN
6. Marks PPA as Valid

```
92      /* new write */  
93      ppa = get_new_page(ssd);  
94      /* update maptbl */  
95      set_maptbl_ent(ssd, lpn, &ppa);  
96      /* update rmap */  
97      set_rmap_ent(ssd, lpn, &ppa);  
98  
99      mark_page_valid(ssd, &ppa);
```



2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★

- *ssd_write()* - Each LPN:

- 7. Advances to Next Write Pointer

- 8. Measures Latency &
Calls Physical NAND Write

```
108      /* need to advance the write pointer here */
109      ssd_advance_write_pointer(ssd);
110
111      struct nand_cmd swr;
112      swr.type = USER_IO;
113      swr.cmd = NAND_WRITE;
114      swr.stime = req->stime;
115      /* get latency statistics */
116      curlat = ssd_advance_status(ssd, &ppa, &swr);
117      maxlat = (curlat > maxlat) ? curlat : maxlat;
118  }
119
120  return maxlat;
121 }
```

2.3. Data Allocation & I/O Processing

■ ftl-datapath.c ★

■ *ssd_trim()* - Each LPN:

1. Returns PPA
2. Marks PPA as Invalid
(If That PPA is Valid)
3. Updates MT that PPA
to INVALID_LPN
4. Updates RMT that LPN
to UNMAPPED_PPA

```
ftl-datapath.c 3 X  ftl-line-gc.c 3  h ftl.h 2  7
ftl-datapath.c > ssd_trim(ssd *, NvmeRequest *)
123  uint64_t ssd_trim(struct ssd *ssd, NvmeRequest *req)
141      for (int range_idx = 0; range_idx < nr_ranges; range_idx++) {
164          // Process each LPN in this range
165          for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
166              ppa = get_maptbl_ent(ssd, lpn);
167
168              // Skip already unmapped/invalid pages
169              if (!mapped_ppa(&ppa) || !valid_ppa(ssd, &ppa)) {
170                  already_invalid++;
171                  continue;
172              }
173
174              // Invalidate the existing mapped page
175              if (exp_lpn_watched(lpn))
176                  EXP_LOG("[INVALIDATE:trim] lpn=%lu old " PPA_FMT "\n",
177                          lpn, PPA_ARG(&ppa));
178              mark_page_invalid(ssd, &ppa);
179
180              // Clear reverse mapping
181              set_rmap_ent(ssd, INVALID_LPN, &ppa);
182
183              // Set mapping table entry as unmapped
184              ppa.ppa = UNMAPPED_PPA;
185              set_maptbl_ent(ssd, lpn, &ppa);
186
187              trimmed_pages++;
188          }
189      }
```

2.3. Data Allocation & I/O Processing

- ftl-datapath.c ★
 - *ssd_trim()*
 - Just Unmapping
 - FTL Does NOT Erase NAND Page Immediately

```
ftl-datapath.c 3 ×  ftl-line-gc.c 3  ftl.h 2
ftl-datapath.c >  ssd_trim(ssd *, NvmeRequest *)
123  uint64_t ssd_trim(struct ssd *ssd, NvmeRequest *req)
141      for (int range_idx = 0; range_idx < nr_ranges; range_idx++) {
164          // Process each LPN in this range
165          for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
166              ppa = get_maptbl_ent(ssd, lpn);
167
168              // Skip already unmapped/invalid pages
169              if (!mapped_ppa(&ppa) || !valid_ppa(ssd, &ppa)) {
170                  already_invalid++;
171                  continue;
172              }
173
174              // Invalidate the existing mapped page
175              if (exp_lpn_watched(lpn))
176                  EXP_LOG("[INVALIDATE:trim] lpn=%lu old " PPA_FMT "\n",
177                          lpn, PPA_ARG(&ppa));
178              mark_page_invalid(ssd, &ppa);
179
180              // Clear reverse mapping
181              set_rmap_ent(ssd, INVALID_LPN, &ppa);
182
183              // Set mapping table entry as unmapped
184              ppa.ppa = UNMAPPED_PPA;
185              set_maptbl_ent(ssd, lpn, &ppa);
186
187              trimmed_pages++;
188          }
}
```

2.3. Data Allocation & I/O Processing

- ftl-media.c
 - Bridge Layer that Returns NAND Flash's R/W/E Latency
 - Does NOT Calculate Latency
(→ hw/femu/nand/nand-media.c)

```
ftl-datapath.c 3  ftl-media.c 3 X  ftl-line-gc.c
ftl-media.c > ...
91  uint64_t ssd_advance_status(struct ssd *ssd, struct ppa *ppa,
92                               struct nand_cmd *ncmd)
93  {
94      uint64_t stime = (ncmd->stime == 0) ?
95                      qemu_clock_get_ns(QEMU_CLOCK_REALTIME) : ncmd->stime;
96      NandLoc loc;
97      NandMediaOp op;
98
99      switch (ncmd->cmd) {
100         case NAND_READ:  op = NAND_MEDIA_READ;    break;
101         case NAND_WRITE: op = NAND_MEDIA_PROGRAM; break;
102         case NAND_ERASE: op = NAND_MEDIA_ERASE;   break;
103         default:
104             ftl_err("Unsupported NAND command: 0x%x\n", ncmd->cmd);
105             return 0;
106         }
107         loc = bb_decode_loc(ssd, ppa);
108         return nand_media_op(&ssd->media, &loc, op, stime).latency_ns;
109     }
110
```

2.4. GC & Wear-leveling

- ftl-line-gc.c ★
 - *mark_page_invalid(PPA)*
 0. If Overwrite or Erase Occurs
 1. Updates Status of
Old & Valid Page to PG_INVALID

```
ftl-line-gc.c 3 X
ftl-line-gc.c > ...
177  /* update SSD status about one page from PG_VALID -> PG_INVALID */
178  void mark_page_invalid(struct ssd *ssd, struct ppa *ppa)
179  {
180      struct line_mgmt *lm = &ssd->lm;
181      struct ssdparams *spp = &ssd->sp;
182      struct nand_block *blk = NULL;
183      struct nand_page *pg = NULL;
184      bool was_full_line = false;
185      struct line *line;
186
187      /* update corresponding page status */
188      pg = get_pg(ssd, ppa);
189      ftl_assert(pg->status == PG_VALID);
190      pg->status = PG_INVALID;
```

2.4. GC & Wear-leveling

■ ftl-line-gc.c ★

■ *mark_page_invalid(PPA)*

2. Updates *vpc* & *ipc* of Block/Line

- *vpc--* : Decreases Valid Page Count
- *ipc++* : Increases Invalid Page Count

→ Crucial in Selecting GC Victim Line

```
/* update corresponding block status */
blk = get_blk(ssd, ppa);
ftl_assert(blk->ipc >= 0 && blk->ipc < spp->pgs_per_blk);
blk->ipc++;
ftl_assert(blk->vpc > 0 && blk->vpc <= spp->pgs_per_blk);
blk->vpc--;

/* update corresponding line status */
line = get_line(ssd, ppa);
ftl_assert(line->ipc >= 0 && line->ipc < spp->pgs_per_line);
if (line->vpc == spp->pgs_per_line) {
    ftl_assert(line->ipc == 0);
    was_full_line = true;
}
line->ipc++;
ftl_assert(line->vpc > 0 && line->vpc <= spp->pgs_per_line);
/* Adjust the position of the victim line in the pq under over-writes */
if (line->pos) {
    /* Note that line->vpc will be updated by this call */
    pqueue_change_priority(lm->victim_line_pq, line->vpc - 1, line);
} else {
    line->vpc--;
}
```

2.4. GC & Wear-leveling

- ftl-line-gc.c ★

- *do_gc()*

1. *select_victim_line()*:

Selects **Victim Line**

By **Lowest VPC Line** (=Greedy)

```
ftl-line-gc.c 3 X
ftl-line-gc.c > ...

382 int do_gc(struct ssd *ssd, bool force)
383 {
384     struct line *victim_line = NULL;
385     struct ssdparams *spp = &ssd->sp;
386     struct nand_lun *lunp;
387     struct ppa ppa;
388     int ch, lun;
389
390     victim_line = select_victim_line(ssd, force);
391     if (!victim_line) {
392         return -1;
393     }
394
395     ppa.g.blk = victim_line->id;
```

2.4. GC & Wear-leveling

- ftl-line-gc.c ★
 - *do_gc()* - Each Block:
 2. *clean_one_block(PPA)* :
 - Read & **Move Valid Page**
by *gc_read_page()*, *gc_write_page()*
(Should NOT be Any Free Page in Victim Block)

```
/* copy back valid data */  
for (ch = 0; ch < spp->nchs; ch++) {  
    for (lun = 0; lun < spp->luns_per_ch; lun++) {  
        ppa.g.ch = ch;  
        ppa.g.lun = lun;  
        ppa.g.pl = 0;  
        lunp = get_lun(ssd, &ppa);  
        clean_one_block(ssd, &ppa);  
        mark_block_free(ssd, &ppa);  
    }  
}
```



2.4. GC & Wear-leveling

■ ftl-line-gc.c ★

■ *do_gc()* - Each Block:

2. *clean_one_block(PPA)* :

- Read & **Move Valid Page**
by *gc_read_page()*, *gc_write_page()*
(Should NOT be Any Free Page in Victim Block)

3. *mark_block_free(PPA)* :

- Updates Status of Page
to **PG_FREE**
- Resets VPC, IPC
- **Increases Erase Count**

```
/* copy back valid data */  
for (ch = 0; ch < spp->nchs; ch++) {  
    for (lun = 0; lun < spp->luns_per_ch; lun++) {  
        ppa.g.ch = ch;  
        ppa.g.lun = lun;  
        ppa.g.pl = 0;  
        lunp = get_lun(ssd, &ppa);  
        clean_one_block(ssd, &ppa);  
        mark_block_free(ssd, &ppa);  
    }  
}
```

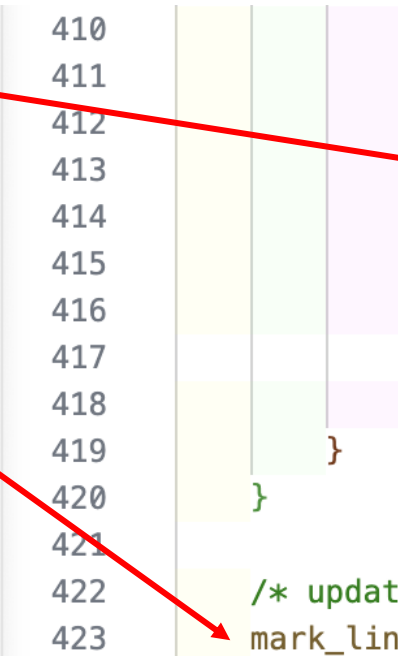
2.4. GC & Wear-leveling

- ftl-line-gc.c ★

- *do_gc()* - Each Block:

4. Calls Physical NAND Erase
5. *mark_line_free(PPA)* :
Updates Victim Line to
Free Line for Rewritable

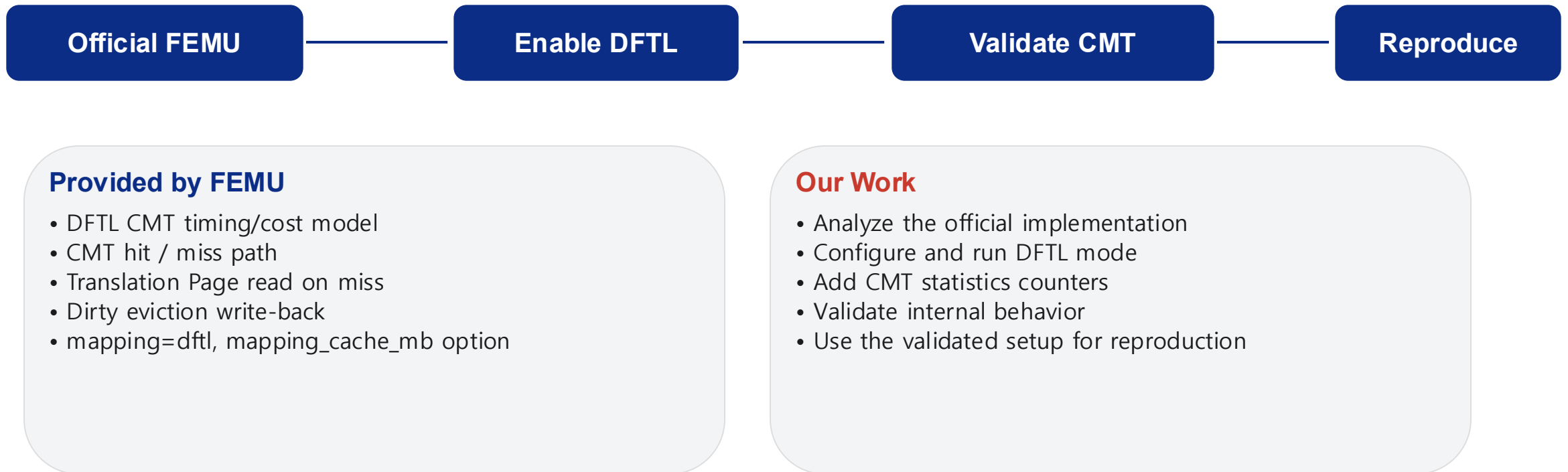
```
410     if (spp->enable_gc_delay) {  
411         struct nand_cmd gce;  
412         gce.type = GC_IO;  
413         gce.cmd = NAND_ERASE;  
414         gce.stime = 0;  
415         ssd_advance_status(ssd, &ppa, &gce);  
416     }  
417  
418     lunp->gc_endtime = lunp->next_lun_avail_time;  
419 }  
420  
421  
422     /* update line status */  
423     mark_line_free(ssd, &ppa);
```



The diagram illustrates the execution flow of the code. It features three vertical bars: a yellow bar on the left, a green bar in the middle, and a purple bar on the right. Red arrows indicate the sequence of execution: one arrow points from the 'Calls Physical NAND Erase' step to the NAND_ERASE command in the code, and another arrow points from the 'Updates Victim Line to Free Line for Rewritable' step to the mark_line_free function call.

3. Reproducing DFTL in FEMU

■ DFTL Support in FEMU



Key point: We did not replace FEMU's FTL; we used FEMU's DFTL model and added observability for validation.

3. Reproducing DFTL in FEMU

■ How FEMU Emulates an SSD



**FEMU stores real data in host DRAM,
but models SSD structure, page states, timing, and GC in software.**

3. Reproducing DFTL in FEMU

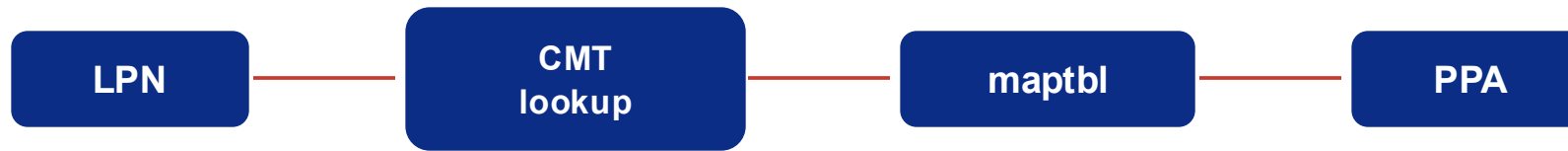
■ DFTL Timing Model in FEMU

Original Page Mapping



Full DRAM L2P mapping
No translation-page I/O

DFTL Mode



HIT

No Translation Page I/O

MISS

Translation Page READ
Dirty eviction → Translation Page WRITE

Functional mapping stays in maptbl; CMT models the additional translation cost.

3. Reproducing DFTL in FEMU

■ Code-Level Change: CMT before Mapping Lookup

Before: Page Mapping

```
/* original page mapping path */  
ppa = ssd->mapping->translate(ssd, lpn);  
/* then data NAND READ/WRITE timing */
```

After: DFTL Mode

```
if (ssd->cmt.capacity) {  
    cmt_touch(ssd, lpn, req->stime, is_write);  
}  
ppa = ssd->mapping->translate(ssd, lpn);
```

CMT HIT

No extra TP I/O
Continue to maptbl

CMT MISS

Issue NAND_READ
Use existing pg_rd_lat

Dirty Eviction

Issue NAND_WRITE
Use existing pg_wr_lat

The DFTL code does not add a fixed constant; it injects TP NAND operations into FEMU's existing timing model.

3. Reproducing DFTL in FEMU

- DFTL CMT Hit/Miss and Translation Page I/O
 - cmt_touch()

```
slot = cmt_lookup(ssd, tp_id);
if (slot >= 0) { // CMT HIT
    ssd->cmt.hits++;
    return 0;
}
ssd->cmt.misses++; // CMT MISS
victim = cmt_clock_evict(ssd);
if (ssd->cmt.slots[victim].dirty)
    ssd_advance_status(... NAND_WRITE ...);
ssd_advance_status(... NAND_READ ...);
```

HIT

No extra Translation Page I/O

MISS

Translation Page READ

Dirty eviction

Translation Page WRITE

File: hw/femu/bbssd/ftl-map-cmt.c · FEMU reuses its existing NAND timing model via ssd_advance_status().

3. Reproducing DFTL in FEMU

■ Our Patch: CMT Instrumentation

Modified Files

ftl.h
ftl-map-cmt.c

```
uint64_t evictions;  
uint64_t dirty_evictions;  
uint64_t tp_reads;  
uint64_t tp_writes;
```

Counting Points

eviction → evictions++
dirty victim → dirty_evictions++
TP READ → tp_reads++
TP WRITE → tp_writes++

Original behavior

→ NAND_READ CMT MISS
Dirty Eviction
→ NAND_WRITE

After patch

→ NAND_READ → tp_reads++ CMT MISS
Dirty Eviction
→ NAND_WRITE → tp_writes++

Patch goal: observe CMT miss, eviction, and Translation Page I/O counts for validation.

3. Reproducing DFTL in FEMU

■ Validation Setup: Why This Workload?

fio Validation Workload

Target: /dev/nvme0n1

Pattern: 4 KB Random Write

Working set: 10 GB

I/O depth: 64

Jobs: 4

Runtime: 30 s

DFTL config: mapping=dftl, CMT=1 MB

Reason 1 — 4 KB random write

One I/O roughly maps to one LPN. Random access touches many Translation Pages.

Reason 2 — write workload

Write requests mark CMT entries dirty, enabling dirty eviction / T P write-back validation.

Reason 3 — 10 GB > 1 MB CMT

The working set is much larger than CMT capacity, so hit ratio and eviction behavior are predictable.

Expected hit ratio: 256 cached TP / 5,120 working-set TP $\approx$ 5%

3. Reproducing DFTL in FEMU

■ Validation Results: CMT Behavior

```
FEMU] DFTL-CMT: accesses=1400000 hits=71382 misses=1328618 evictions=1328362 dirty_evictions=1328359 tp_reads=1328618 tp_writes=1328359 used=256/256
FEMU] DFTL-CMT: accesses=1500000 hits=76450 misses=1423550 evictions=1423294 dirty_evictions=1423291 tp_reads=1423550 tp_writes=1423291 used=256/256
FEMU] DFTL-CMT: accesses=1600000 hits=81383 misses=1518617 evictions=1518361 dirty_evictions=1518358 tp_reads=1518617 tp_writes=1518358 used=256/256
FEMU] DFTL-CMT: accesses=1700000 hits=86401 misses=1613599 evictions=1613343 dirty_evictions=1613340 tp_reads=1613599 tp_writes=1613340 used=256/256
FEMU] DFTL-CMT: accesses=1800000 hits=91379 misses=1708621 evictions=1708365 dirty_evictions=1708362 tp_reads=1708621 tp_writes=1708362 used=256/256
FEMU] DFTL-CMT: accesses=1900000 hits=96450 misses=1803550 evictions=1803294 dirty_evictions=1803291 tp_reads=1803550 tp_writes=1803291 used=256/256
FEMU] DFTL-CMT: accesses=2000000 hits=101480 misses=1898520 evictions=1898264 dirty_evictions=1898261 tp_reads=1898520 tp_writes=1898261 used=256/256
un@sslab:~/tmp/FEMU_jinwook/build-femu$
```

MISS → TP READ

$\text{misses} = \text{tp_reads}$
 $1,898,520 = 1,898,520$

Dirty Eviction → TP WRITE

$\text{dirty_evictions} = \text{tp_writes}$
 $1,898,261 = 1,898,261$

Eviction Count

$\text{misses} - \text{capacity} = \text{evictions}$
 $1,898,520 - 256 = 1,898,264$

Measured Hit Ratio

$101,480 / 2,000,000 = 5.07\%$

Expected Hit Ratio

$256 / 5,120 \approx 5.00\%$

Conclusion: the internal DFTL-CMT timing behavior matched the implementation logic.

3. Reproducing DFTL in FEMU

■ Baseline Page Mapping vs DFTL (Fig. 9 in DFTL Paper)

Observed performance impact

IOPS (K)



Bandwidth (MiB/s)



Average latency (ms)

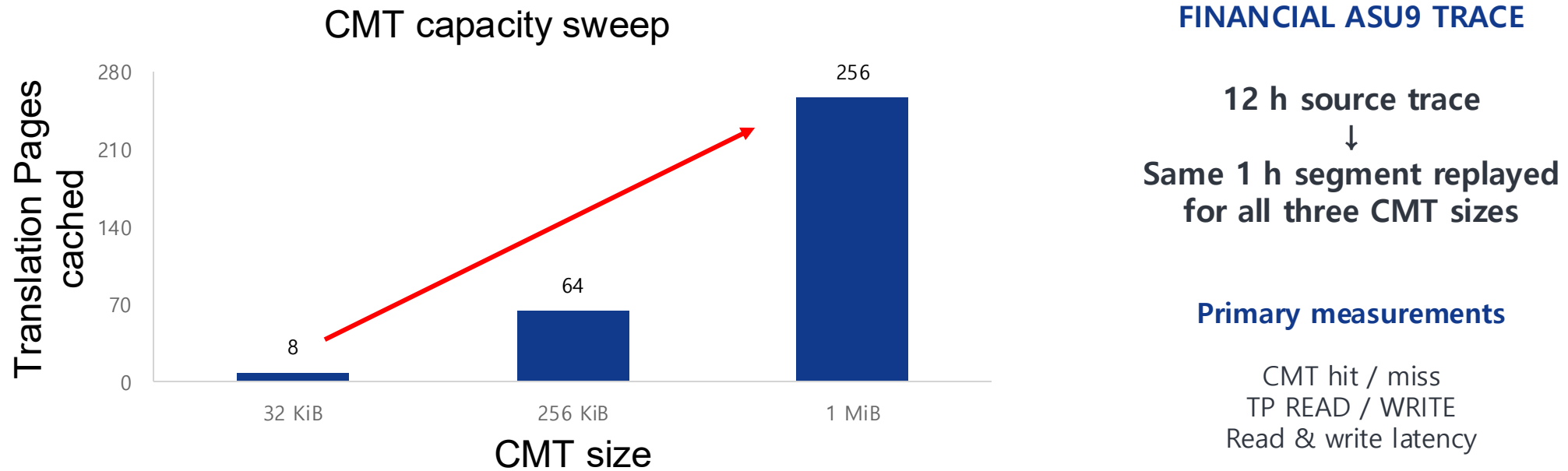


Interpretation

DFTL has additional Translation Page I/O on CMT misses and dirty evictions. This increases latency and reduces throughput.

3. Reproducing DFTL in FEMU

- CMT Size Sweep with Financial Trace (Fig. 11(a) in DFTL Paper)

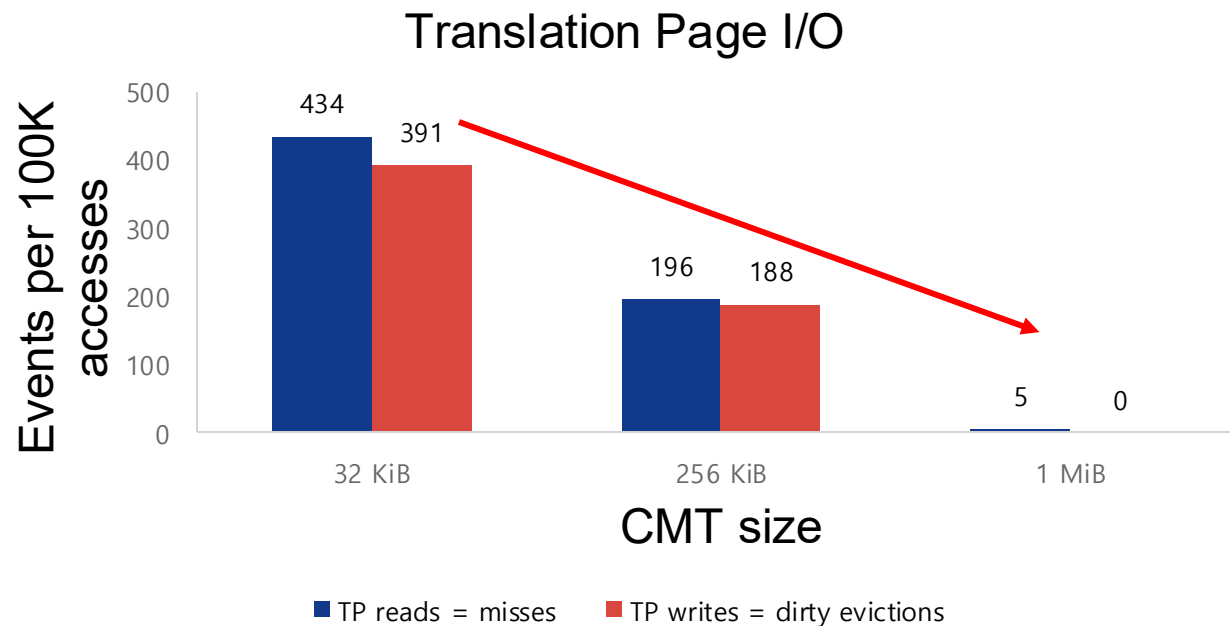


Does a larger CMT reduce Translation Page I/O and improve latency?

Scope: the measured one-hour segment should not be generalized as the full 12-hour trace.

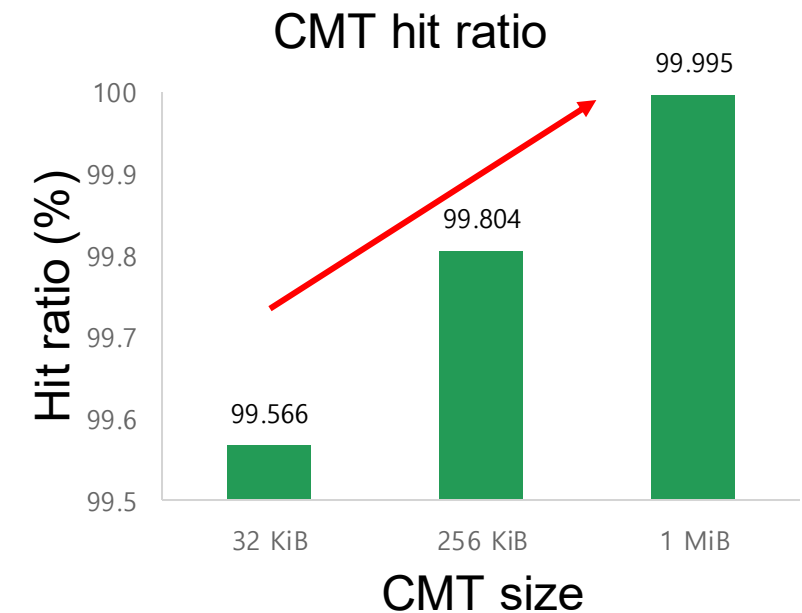
3. Reproducing DFTL in FEMU

- Larger CMT Reduces Translation Page I/O (Fig. 11(a) in DFTL Paper)



Misses / TP reads: 434 → 196 → 5
Dirty evictions / TP writes: 391 → 188 → 0

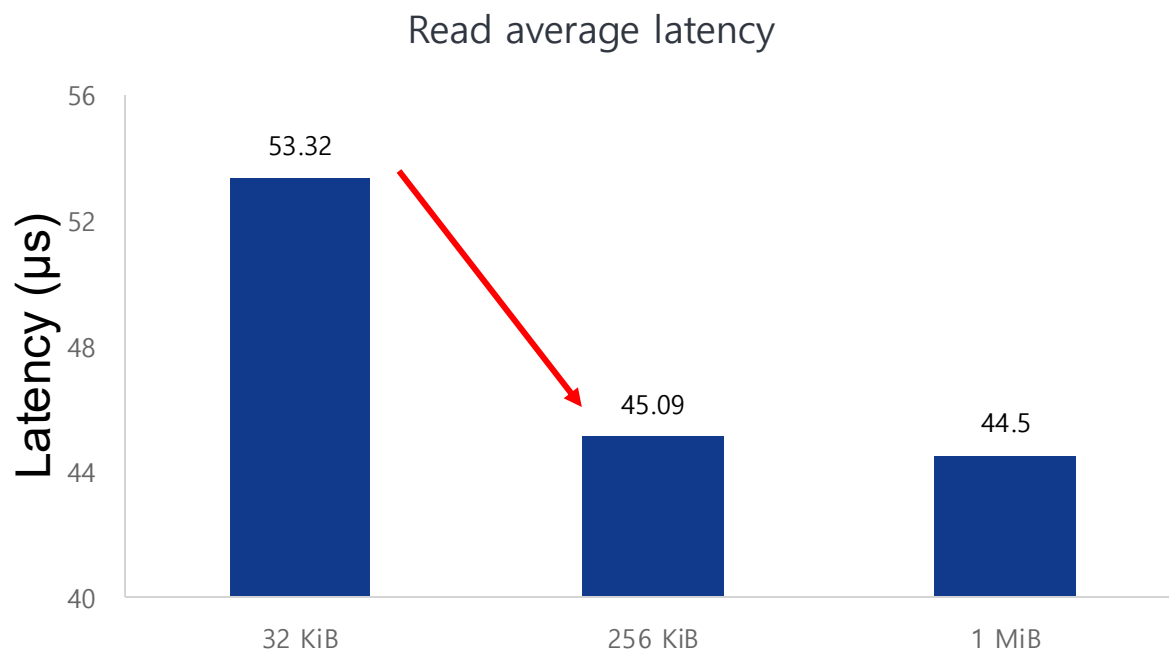
Larger CMT → fewer misses → fewer Translation Page NAND I/Os



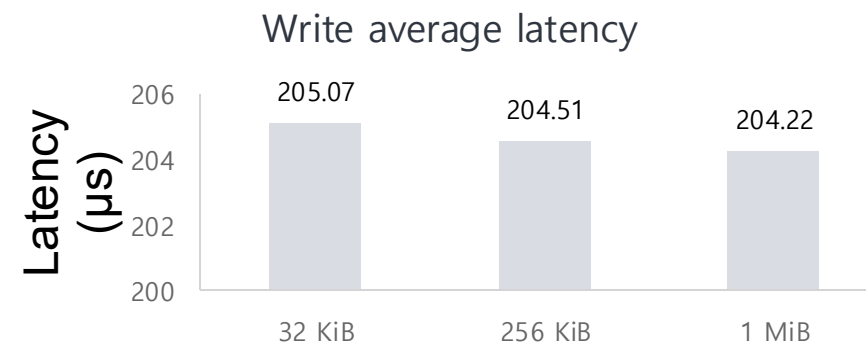
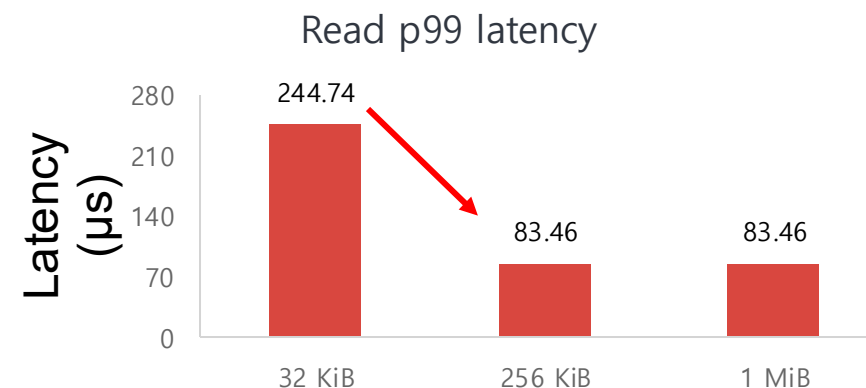
1 MiB used 214 → 219 of 256 slots.
CMT never filled, so eviction and write-back were zero.

3. Reproducing DFTL in FEMU

■ Performance Impact of CMT Size



Read average: 53.32 → 44.50 μs (≈16.5% reduction)



Most improvement: 32 KiB → 256 KiB | Diminishing returns: 256 KiB → 1 MiB

Trace replay constrains request arrival rate; latency and CMT statistics are the primary metrics.

Conclusion

01

Model analyzed

Official FEMU DFTL-CMT timing behavior was traced and instrumented.

02

Internal invariants validated

misses = TP reads; dirty evictions = TP writes; expected cache behavior matched.

03

CMT trade-off reproduced

Financial trace: larger CMT reduced TP I/O and read latency, with diminishing returns.

04

Scope remains limited

Functional mapping still uses flat maptbl; this is not full original-DFTL fidelity.

Outcome: an experimental environment that makes FEMU's core CMT cost/benefit observable and reproducible.

Thank you & Any Questions?

Appendix

■ Event Counting Points

Instrumented event path

```
if (ssd->cmt.slots[victim].valid) {  
    ssd->cmt.evictions++;  
  
    if (ssd->cmt.slots[victim].dirty) {  
        ssd->cmt.dirty_evictions++;  
        wb_lat = ssd_advance_status(ssd, &vppa, &twr);  
        ssd->cmt.tp_writes++;  
    }  
}  
  
lat = wb_lat + ssd_advance_status(ssd, &tpa, &trd);  
ssd->cmt.tp_reads++;
```

Victim replaced

→ evictions++

Dirty victim

→ dirty_evictions++

TP NAND WRITE

→ tp_writes++

TP NAND READ → tp_reads++

Appendix

■ Why Is the Expected Hit Ratio $\approx 5\%$?

Four-step calculation

- 1 Translation Page coverage
 $4 \text{ KiB} / 8 \text{ B} = 512 \text{ mappings / TP}$
 $512 \text{ LPN} \times 4 \text{ KiB} = 2 \text{ MiB logical data / TP}$
- 2 CMT capacity
 $1 \text{ MiB} / 4 \text{ KiB} = 256 \text{ TP}$
- 3 Working-set Translation Pages
 $10 \text{ GiB} / 2 \text{ MiB} = 5,120 \text{ TP}$
- 4 Expected Hit Ratio
 $256 / 5,120 = 0.05 \approx 5\%$

Working Set — 5,120 TPs

10 GiB spans 5,120 Translation Pages.
CMT can hold only 256 of them.

CMT — 256 TPs

256 cached TPs + 4,864 uncached TPs
= 5,120 working-set TPs

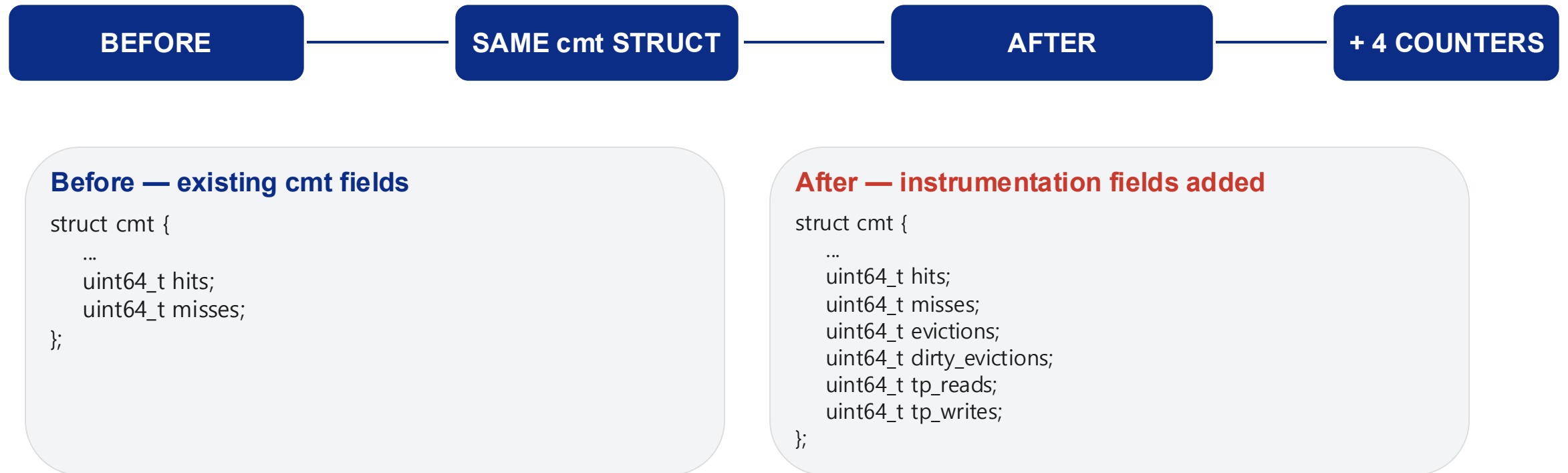
Expected vs. measured

Expected: 5.00% | Measured: 5.07%
The measured result closely matches the model.

Assumption: approximately uniform random access over the 10 GiB working set.

Appendix

■ Instrumentation Patch — Counter Declarations



Added observability counters without changing the DFTL algorithm.

Appendix

■ hw/femu/bbssd/ftl.c

```
/*
 * L2P mapping scheme: "page" (default, full DRAM L2P, bit-identical) vs "dftl"
 * (demand-cached L2P via the CMT). The CMT turns on when dftl is named or
 * mapping_cache_mb is set; a named dftl with no explicit size gets 4 MB.
 */
ssd->mapping = femu_mapping_scheme_lookup(n->bb_params.mapping_scheme);
ssd->map_priv = NULL;
uint32_t map_cache_mb = n->mapping_cache_mb;
if (femu_mapping_scheme_uses_cmt(ssd->mapping) && map_cache_mb == 0) {
    map_cache_mb = 4; /* default dftl cache when no explicit size given */
}
/* the CMT is a dftl-only cost model: never charge it under the page scheme,
 * even if mapping_cache_mb was set */
cmt_init(ssd, femu_mapping_scheme_uses_cmt(ssd->mapping) ? map_cache_mb : 0);
```

왜 필요한가?

코드 재빌드 없이 CMT 크기를 1MB, 2MB, 4MB, 8MB로 바꿔 실험할 수 있다.

1. 실행 인자
mapping=dftl
mapping_cache_mb=1

2. FTL 초기화
dftl scheme 선택

3. CMT 생성
1MiB → 256 TP

Appendix

- hw/femu/bbssd/ftl-map.c

```
static const struct femu_mapping_ops femu_mapping_schemes[] = {  
    {  
        .name = "page",  
        .uses_cmt = false,  
        .translate = femu_map_page_translate,  
        .prepare_write = femu_map_page_prepare_write,  
        .commit_write = femu_map_page_commit_write,  
        .gc_relocate_commit = femu_map_page_gc_relocate_commit,  
    },  
    {  
        .name = "dftl",  
        .uses_cmt = true,  
        .translate = femu_map_page_translate,  
        .prepare_write = femu_map_page_prepare_write,  
        .commit_write = femu_map_page_commit_write,  
        .gc_relocate_commit = femu_map_page_gc_relocate_commit,  
    },  
};
```

page는 CMT 사용 X
'uses_cmt = false'

dftl은 CMT를 사용 O
'uses_cmt = true'

Appendix

■ hw/femu/bbssd/ftl-datapath.c

```
/* normal IO read path */
for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
    /* dump the FEMU_DUMP_LPN page on every read (config read once at init) */
    if (exp_dump_lpn_set && lpn == exp_dump_lpn)
        femu_dbg_dump_lpn(ssd, lpn);

    /* dftl: charge the demand-cache translation cost (no-op when disabled) */
    if (ssd->cmt.capacity) {
        uint64_t clat = cmt_touch(ssd, lpn, req->stime, false);
        maxlat = (clat > maxlat) ? clat : maxlat;
    }
}
```

기존에는 LPN을 받으면 바로
mapping table을 조회

DFTL에서는 그 앞에 CMT 조회 비용
을 먼저 계산한다.

기존 I/O 경로 앞에 cmt_touch()를 삽입

```
for (lpn = start_lpn; lpn <= end_lpn; lpn++) {
    /* dftl: charge the demand-cache translation cost (no-op when disabled) */
    if (ssd->cmt.capacity) {
        uint64_t clat = cmt_touch(ssd, lpn, req->stime, true);
        maxlat = (clat > maxlat) ? clat : maxlat;
    }
}
```

Appendix

■ hw/femu/bbssd/ftl-map-cmt.c

```
uint64_t cmt_touch(struct ssd *ssd, uint64_t lpn, uint64_t stime, bool is_write)
{
    struct ssdparams *spp = &ssd->sp;
    uint64_t tp_id = lpn / ssd->cmt.lpn_per_tp;
    int32_t slot = cmt_lookup(ssd, tp_id);
    uint64_t lat = 0;

    if (slot >= 0) {
        ssd->cmt.slots[slot].ref = true;
        if (is_write) {
            ssd->cmt.slots[slot].dirty = true;
        }
        ssd->cmt.hits++;
        return 0;
    }
}
```

HIT 경로

필요한 Translation Page가 CMT에 있으면 바로 끝난다.
WRITE라면 dirty만 표시한다.

MISS 경로

필요한 TP가 없으면 NAND_READ를 발생시킨다.
CMT가 꽉 차서 dirty victim을 버리면 먼저 NAND_WRITE가 발생한다

```
uint64_t wb_lat = 0;
if (ssd->cmt.slots[victim].valid) {
    if (ssd->cmt.slots[victim].dirty) {
        uint32_t vhome = ssd->cmt.slots[victim].tp_id % spp->tt_luns;
        struct ppa vppa = {.ppa = 0};
        vppa.g.ch = vhome / spp->luns_per_ch;
        vppa.g.lun = vhome % spp->luns_per_ch;
        struct nand_cmd twr = {.type = USER_IO, .cmd = NAND_WRITE,
                               .stime = stime};
        wb_lat = ssd_advance_status(ssd, &vppa, &twr);
    }
    ssd->cmt.used--;

    uint32_t home = tp_id % spp->tt_luns;
    struct ppa tppa = {.ppa = 0};
    tppa.g.ch = home / spp->luns_per_ch;
    tppa.g.lun = home % spp->luns_per_ch;
    struct nand_cmd trd = {.type = USER_IO, .cmd = NAND_READ,
                           .stime = stime + wb_lat};
}
```

Appendix

■ hw/femu/bbssd/ftl.h

Original behavior

```
struct {
    uint32_t capacity; /* number of cached pages (0 = off) */
    uint32_t used;
    uint32_t hand; /* CLOCK hand */
    uint32_t evict_policy; /* 0 = CLOCK, 1 = random, 2 = LRU, 3 = 2Q */
    uint64_t rand_state; /* LCG state for reproducible random eviction */
    uint64_t tick; /* monotonic recency counter for LRU */
    uint64_t hit_lat; /* DRAM read-hit latency (ns) */
    uint64_t hits;
    uint64_t misses;
    struct rcache_slot *slots;
    int32_t *hash; /* lpn hash -> slot idx, -1 empty */
    uint32_t hash_sz;
} rcache;
```

After patch

```
struct {
    uint32_t capacity; /* cached translation pages (0 = off) */
    uint32_t used;
    uint32_t hand; /* CLOCK hand */
    uint32_t lpn_per_tp; /* LPNs covered by one translation page */
    uint64_t hits;
    uint64_t misses;
    uint64_t evictions; /* valid CMT entries replaced */
    uint64_t dirty_evictions; /* evicted entries that require TP write-back */
    uint64_t tp_reads; /* translation-page NAND reads */
    uint64_t tp_writes; /* translation-page NAND write-backs */
    struct cmt_slot *slots;
    int32_t *hash; /* tp_id hash -> slot idx, -1 empty */
    uint32_t hash_sz;
} cmt;
```


Appendix

- hw/femu/bbssd/ftl-map-cmt.c

```
static inline void cmt_maybe_report_stats(struct ssd *ssd)
{
    uint64_t accesses = ssd->cmt.hits + ssd->cmt.misses;

    if (accesses && accesses % CMT_STATS_REPORT_INTERVAL == 0) {
        fprintf(stderr, "[FEMU] DFTL-CMT: accesses=%"PRIu64
            " hits=%"PRIu64" misses=%"PRIu64
            " evictions=%"PRIu64" dirty_evictions=%"PRIu64
            " tp_reads=%"PRIu64" tp_writes=%"PRIu64
            " used=%u/%u\n",
            accesses, ssd->cmt.hits, ssd->cmt.misses,
            ssd->cmt.evictions, ssd->cmt.dirty_evictions,
            ssd->cmt.tp_reads, ssd->cmt.tp_writes,
            ssd->cmt.used, ssd->cmt.capacity);
    }
}
```